TP – Conception Orientée Objet

CI1 FISA ITR — projet en trinôme

L'association IMT Arena organise sa saison de tournois e-sport. Au programme : des équipes qui s'affrontent, des coachs qui hurlent depuis le bord de l'écran, des arbitres qui ne servent à rien mais qui sifflent quand même, et une salle qui sent le café froid. Pour gérer tout cela, l'association fait appel aux étudiants de FISA ITR, dont la rumeur dit que la conception objet est la plus aboutie du Nord. Votre mission : concevoir et réaliser la librairie d'objets qui permettra de planifier, simuler et archiver les tournois. Le langage support est celui de votre choix (tant qu'il répond aux notions liées à la programmation objet). Tout Easter Egg est bienvenu, du moment que le serveur ne plante pas en finale.

1. Ce qu'il faut modéliser

Les personnes. IMT Arena gère trois catégories de personnes, toutes identifiées par un id, un nom, un prénom, un pseudo, un genre et un téléphone :

- **Les joueurs** appartiennent à une seule équipe à la fois. Ils ont un statut, amateur ou pro. Un joueur amateur devient pro quand son équipe a gagné 3 matchs depuis qu'il y est.
- **Les coachs** dirigent une seule équipe à la fois. On connaît le nombre total de matchs gagnés par les équipes qu'ils ont coachées durant leur carrière.
- **Les arbitres** ne jouent jamais. Ils sont qualifiés pour un ou plusieurs jeux et on sait combien de matchs ils ont arbitré.

Les jeux. Un jeu a un nom et un nombre de manches par match. On distingue :

- **Les jeux en solo** : une équipe aligne un seul joueur par match (jeux de combat, de cartes…). Le joueur est désigné parmi les joueurs de l'équipe.
- **Les jeux en équipe** : une équipe aligne plusieurs de ses joueurs (de 3 à 5, selon le jeu).

Certaines capacités sont communes à plusieurs objets sans être de même nature (être classé, être qualifié pour un jeu…) : à vous de décider comment les exprimer. Votre conception doit permettre d'ajouter un nouveau type de jeu sans modifier l'existant.

Les équipes et les matchs. Une équipe a un nom, un coach, des joueurs et un total de points : une victoire en rapporte 3, un match nul n'existe pas. Un match oppose deux équipes différentes sur un jeu, sous le contrôle d'un arbitre qualifié pour ce jeu. Il garde la trace des deux équipes, de l'arbitre et du vainqueur.

Les tournois. Un tournoi a un nom, un jeu, une date, un budget (donné, pas calculé), un nombre d'arènes (les matchs peuvent se jouer en parallèle, de 1 à 4 arènes) et des équipes inscrites. Dans un tournoi, chaque équipe rencontre chaque autre équipe une fois (6 équipes : 15 matchs). Il est planifié, en cours ou terminé.

- Pour démarrer : au moins 4 équipes, chacune avec un coach et le bon nombre de joueurs pour le jeu, au moins un arbitre qualifié, et aucun joueur, coach ou équipe déjà engagé dans un tournoi en cours.
- Plusieurs tournois peuvent être planifiés. Plusieurs peuvent être en cours à la fois (max 3), et celui qu'on lance est toujours le plus proche dans le temps.
- Un tournoi est réussi si au moins 70 % de ses matchs ont pu être joués (les autres ont subi un incident technique).
- La fin de tournoi est automatique : points des équipes, victoires des coachs, matchs arbitrés, promotions des joueurs, création du rapport de tournoi (nom, jeu, matchs joués, vainqueur, réussite).

2. La simulation d'un tournoi

Il n'est pas demandé de simuler une partie de jeu : la simulation est pilotée par le temps et par le hasard. Quand un tournoi est lancé, il s'exécute dans un thread à part (géré par un ExecutorService), pour que le programme principal reste utilisable pendant ce temps. Le déroulement :

1. **Durée** — la simulation dure un temps qui dépend de la taille du tournoi : (nombre de matchs × temps par match) ÷ nombre d'arènes du tournoi. Par exemple 15 matchs × 100 ms avec 3 arènes : 500 ms. Le temps par match est une constante réglable, pour que les tests restent rapides.
2. **Matchs** — chaque match reçoit un arbitre qualifié (au hasard ou à tour de rôle). A l'issue de la durée du match, un incident technique est tiré au sort (par exemple 10 % de chances) : le match est alors annulé, sans points. Sinon, le vainqueur est tiré au sort. Le tirage peut favoriser l'équipe la mieux classée.

3. **Fin** — le tournoi passe à "terminé", réussi ou non selon la règle des 70 %, et son rapport est produit.

Le générateur aléatoire (Random, avec une graine fixe dans les tests) rend le résultat reproductible. Contraintes de sûreté :

- Pendant la simulation, le programme principal peut consulter l'état du tournoi (en cours ou terminé) et récupérer son rapport une fois fini (avec un délai maximal d'attente), sans jamais voir de données incohérentes.
- Plusieurs tournois peuvent être en cours à la fois mais maximum 3 : si deux demandes de lancement arrivent en même temps il faut garantir qu'il n'y en ait pas plus que 3.
- La simulation s'arrête proprement : à la fin du programme, le pool est fermé et aucun thread ne continue à tourner.

3. Fonctionnalités attendues

- Ajouter des personnes, des jeux, des équipes, des tournois.
- Planifier des tournois, lancer le suivant, le simuler, le clôturer automatiquement.
- Lister : tournois planifiés (le plus proche d'abord), tournois terminés (du plus gros au plus petit budget), classement des équipes (par points décroissants, puis par nom), arbitres disponibles et occupés, matchs d'une équipe, matchs d'un tournoi regroupés par équipe gagnante.

4. Exigences techniques

Héritage, encapsulation, interfaces. Utilisez-les là où ils ont un intérêt, pas par "plaisir". Les règles ci-dessus (équipes, statuts, qualifications, démarrage d'un tournoi) doivent être garanties par les objets eux-mêmes : on ne doit pas pouvoir créer un état incohérent depuis l'extérieur.

Framework Collection. Pour chacun des besoins suivants, choisissez la structure adaptée (ArrayList ou LinkedList, HashMap, TreeMap ou LinkedHashMap, HashSet, TreeSet ou EnumSet, PriorityQueue…) et utilisez Comparable et Comparator à bon escient :

- tournois planifiés, le plus proche d'abord ;
- classement des équipes, sans doublon, trié par points puis par nom ;
- accès direct à une équipe à partir de son nom ;
- jeux pour lesquels un arbitre est qualifié ;
- matchs d'un tournoi regroupés par équipe gagnante.

Bonnes pratiques de conception. Lorsque c'est adapté, appuyez-vous sur des design patterns vus en cours, et nommez-les dans votre dossier de conception en expliquant le problème qu'ils résolvent. Quelques situations que vous allez probablement rencontrer :

- Le vainqueur d'un match peut être désigné de plusieurs façons (tirage équitable, favorisant la meilleure équipe, toujours la mieux classée pour faciliter les tests…), et on voudrait en changer sans toucher au match.
- Quand un match se termine ou qu'un incident survient, plusieurs éléments doivent en être informés (classement, journal, affichage…), sans que le match les connaisse tous.
- Créer un jeu à partir d'un simple nom, ou construire un tournoi en vérifiant qu'il est valide, ne devrait pas obliger à répéter du code partout.

Aucun pattern n'est obligatoire : mieux vaut un pattern qui se justifie que beaucoup d'artificiels.

Persistance. Seuls les rapports de tournoi sont persistés, dans une base H2 stockée dans un fichier (par exemple jdbc:h2:./data/arena), avec JPA. Le rapport est un objet dédié (@Entity : id, nom du tournoi, date, jeu, nombre de matchs prévus, nombre de matchs joués, nom de l'équipe vainqueur, réussite) créé automatiquement à la fin de chaque tournoi. On doit pouvoir retrouver tous les rapports ou ceux d'un jeu donné.

Tests. Au moins trois tests JUnit 5 sur ce que vous choisissez, en essayant de diversifier les tests (paramétrés, exception, before/after each, etc.).

Les diverses actions sont aussi illustrées dans un programme principal avec un jeu de démonstration. Utilisez des Random (avec une graine fixe dans les tests) pour éviter la saisie de trop de données. Pas de System.out.println en dehors du main et des tests.

5. Bonnes pratiques et conseils

- Concevez avant de coder (diagramme de classes sur papier).
- Projet Maven ou Gradle.
- Les choix doivent rester logiques : pas de match d'une équipe contre elle-même, pas de joueur dans deux équipes, pas de tournoi sans arbitre.
- Peu d'instances suffisent : quatre à six équipes de trois joueurs, quelques arbitres, un ou deux jeux.
- La gestion des id n'est pas la priorité : partez du principe qu'ils sont uniques (ou laissez JPA les générer).
- Le temps étant compté : faites d'abord fonctionner le plus simple (personnes, équipes, tournoi, simulation), puis ajoutez la persistance et les tests.

6. Rendu et évaluation

À rendre sur Moodle, au format ZIP uniquement (tout autre format ou hors délai : note/2) : le projet complet, un court README (comment lancer le main et les tests), et un "dossier" de conception de 2 pages maximum contenant le diagramme de classes, les collections choisies et leurs justifications, les patterns utilisés (s'il y en a) et les choix de synchronisation.

Echange : un entretien de 10 minutes par groupe (un membre différent du trinôme à chaque fois) aux séances 2, 3 et 4.

Critère	Points
Réalisation	8
Echanges	6
Assiduité - ponctualité	2
Qualité du code, du "dossier" de conception et du rendu	4
Total	20

Ce n'est pas seulement "ce qui marche" qui est évalué, mais surtout "comment ce qui fonctionne est conçu". Une application qui tourne mais sans objets, non évolutive, qui ne prend pas en compte les points vus en cours ne rapporte que peu de points.